

Big Data y Machine Learning para Economía Aplicada

Week 04: Árboles, Random Forest y Boosting

Eduard F. Martinez Gonzalez, Ph.D.

Departamento de Economía, Universidad Icesi

July 23, 2026

Previously...

- Tres sesiones dentro del mundo **lineal**: el riesgo y el sobreajuste (S1), cómo medir el error honestamente (S2) y cómo controlar la complejidad encogiendo coeficientes con λ (S2–S3).
- Y el puente causal: el Lasso selecciona *predictores*, no *causas*; para inferencia, doble selección.

Previously...

- Tres sesiones dentro del mundo **lineal**: el riesgo y el sobreajuste (S1), cómo medir el error honestamente (S2) y cómo controlar la complejidad encogiéndolo coeficientes con λ (S2–S3).
- Y el puente causal: el Lasso selecciona *predictores*, no *causas*; para inferencia, doble selección.

Hoy: cambio total de familia

Salimos de los modelos lineales. **Árboles de decisión** y sus dos ensambles:

- **Random Forest**: muchos árboles grandes *en paralelo*, promediados.
- **Boosting**: muchos árboles pequeños *en secuencia*, sumados.

Son los métodos que, en la práctica, más ganan con datos tabulares — y traen una palanca anti-varianza nueva: **promediar**.

Previously...

- Tres sesiones dentro del mundo **lineal**: el riesgo y el sobreajuste (S1), cómo medir el error honestamente (S2) y cómo controlar la complejidad encogiéndolo coeficientes con λ (S2–S3).
- Y el puente causal: el Lasso selecciona *predictores*, no *causas*; para inferencia, doble selección.

Hoy: cambio total de familia

Salimos de los modelos lineales. **Árboles de decisión** y sus dos ensambles:

- **Random Forest**: muchos árboles grandes *en paralelo*, promediados.
- **Boosting**: muchos árboles pequeños *en secuencia*, sumados.

Son los métodos que, en la práctica, más ganan con datos tabulares — y traen una palanca anti-varianza nueva: **promediar**.

La sesión es larga (era dos): CART $\rightarrow$ bagging/RF $\rightarrow$ boosting/XGBoost $\rightarrow$ cuál usar cuándo.

1 Árboles de decisión (CART)

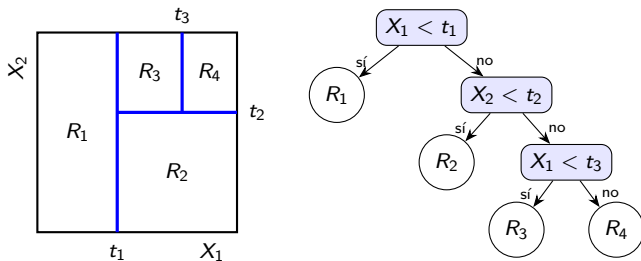
2 Bagging y Random Forest

3 Boosting y XGBoost

4 RF vs. boosting: ¿cuál y cuándo?

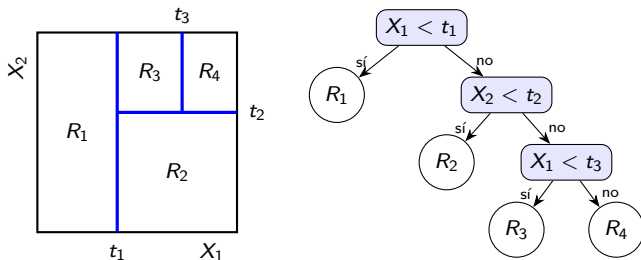
La idea: partir el espacio en rectángulos

Un árbol divide el espacio de predictores con **cortes binarios sucesivos** y predice una **constante** en cada región (la media de y ; o la clase mayoritaria).



La idea: partir el espacio en rectángulos

Un árbol divide el espacio de predictores con **cortes binarios sucesivos** y predice una **constante** en cada región (la media de y ; o la clase mayoritaria).



- Ambos paneles son **el mismo modelo**: cada hoja del árbol es un rectángulo del espacio.
- Predicción en regresión: $\hat{y} = \bar{y}_{R_m}$ para la región donde cae x .

¿Cómo se elige cada corte?

En regresión: en cada paso, buscar el par (variable j , umbral s) que más reduce la suma de cuadrados:

$$\min_{j, s} \sum_{i: x_i \in R_1(j, s)} (y_i - \bar{y}_{R_1})^2 + \sum_{i: x_i \in R_2(j, s)} (y_i - \bar{y}_{R_2})^2$$

con $R_1(j, s) = \{x : x_j < s\}$ y $R_2(j, s) = \{x : x_j \geq s\}$.

¿Cómo se elige cada corte?

En regresión: en cada paso, buscar el par (variable j , umbral s) que más reduce la suma de cuadrados:

$$\min_{j, s} \sum_{i: x_i \in R_1(j, s)} (y_i - \bar{y}_{R_1})^2 + \sum_{i: x_i \in R_2(j, s)} (y_i - \bar{y}_{R_2})^2$$

con $R_1(j, s) = \{x : x_j < s\}$ y $R_2(j, s) = \{x : x_j \geq s\}$.

- La búsqueda es **exhaustiva** sobre (j, s) pero barata: para cada variable basta evaluar los n umbrales entre observaciones consecutivas.
- El procedimiento es **greedy y recursivo** (*top-down*): el mejor corte *ahora*, sin mirar adelante; luego repetir dentro de cada mitad.

¿Cómo se elige cada corte?

En regresión: en cada paso, buscar el par (variable j , umbral s) que más reduce la suma de cuadrados:

$$\min_{j, s} \sum_{i: x_i \in R_1(j, s)} (y_i - \bar{y}_{R_1})^2 + \sum_{i: x_i \in R_2(j, s)} (y_i - \bar{y}_{R_2})^2$$

con $R_1(j, s) = \{x : x_j < s\}$ y $R_2(j, s) = \{x : x_j \geq s\}$.

- La búsqueda es **exhaustiva** sobre (j, s) pero barata: para cada variable basta evaluar los n umbrales entre observaciones consecutivas.
- El procedimiento es **greedy y recursivo** (*top-down*): el mejor corte *ahora*, sin mirar adelante; luego repetir dentro de cada mitad.
- Consecuencias del enfoque greedy: rápido y simple, pero nada garantiza el árbol globalmente óptimo (encontrarlo es NP-difícil).

¿Cómo se elige cada corte?

En regresión: en cada paso, buscar el par (variable j , umbral s) que más reduce la suma de cuadrados:

$$\min_{j, s} \sum_{i: x_i \in R_1(j, s)} (y_i - \bar{y}_{R_1})^2 + \sum_{i: x_i \in R_2(j, s)} (y_i - \bar{y}_{R_2})^2$$

con $R_1(j, s) = \{x : x_j < s\}$ y $R_2(j, s) = \{x : x_j \geq s\}$.

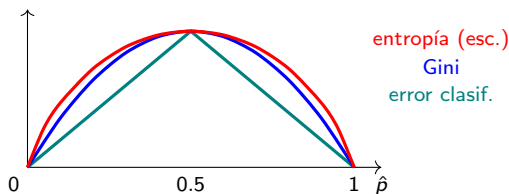
- La búsqueda es **exhaustiva** sobre (j, s) pero barata: para cada variable basta evaluar los n umbrales entre observaciones consecutivas.
- El procedimiento es **greedy y recursivo** (*top-down*): el mejor corte *ahora*, sin mirar adelante; luego repetir dentro de cada mitad.
- Consecuencias del enfoque greedy: rápido y simple, pero nada garantiza el árbol globalmente óptimo (encontrarlo es NP-difícil).

Lo que el árbol hace gratis

No linealidades (funciones escalón) e **interacciones** (cortes anidados condicionan unos en otros) — sin que nadie las especifique. Compárenlo con agregar términos a mano en una regresión.

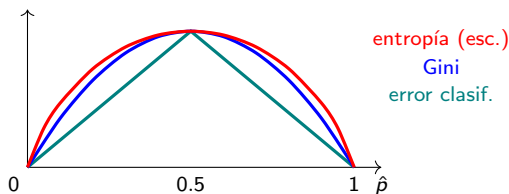
Clasificación: medidas de impureza

Con y categórica, “reducir RSS” se reemplaza por “dejar los nodos más **puros**”.
Para un nodo con proporción $\hat{p}$ de la clase 1 (caso binario):



Clasificación: medidas de impureza

Con y categórica, “reducir RSS” se reemplaza por “dejar los nodos más **puros**”.
Para un nodo con proporción $\hat{p}$ de la clase 1 (caso binario):



- **Error de clasificación:** $1 - \max(\hat{p}, 1 - \hat{p})$. **Gini:** $2\hat{p}(1 - \hat{p})$. **Entropía:** $-\hat{p} \log \hat{p} - (1 - \hat{p}) \log(1 - \hat{p})$.
- Para **crecer** el árbol se usan Gini o entropía: son *estrictamente cóncavas* y diferenciables $\Rightarrow$ premian avances hacia nodos puros que el error (plano por tramos) ni nota.
- Máxima impureza en $\hat{p} = 0.5$ (moneda al aire); cero en nodos puros.

¿Cuándo parar? Crecer grande y podar

El dilema: un árbol que corta hasta dejar una observación por hoja tiene error de entrenamiento cero — la **memorización** de la sesión 1. Parar temprano con reglas locales (“no cortar si la mejora es pequeña”) es miope: un corte mediocre puede habilitar uno excelente después.

¿Cuándo parar? Crecer grande y podar

El dilema: un árbol que corta hasta dejar una observación por hoja tiene error de entrenamiento cero — la **memorización** de la sesión 1. Parar temprano con reglas locales (“no cortar si la mejora es pequeña”) es miope: un corte mediocre puede habilitar uno excelente después.

La solución de CART (Breiman et al., 1984): crecer un árbol grande T_0 y luego **podar** resolviendo

$$\min_{T \subseteq T_0} \sum_{m=1}^{|T|} \sum_{i: x_i \in R_m} (y_i - \bar{y}_{R_m})^2 + \alpha |T|$$

donde $|T|$ es el número de hojas.

¿Cuándo parar? Crecer grande y podar

El dilema: un árbol que corta hasta dejar una observación por hoja tiene error de entrenamiento cero — la **memorización** de la sesión 1. Parar temprano con reglas locales (“no cortar si la mejora es pequeña”) es miope: un corte mediocre puede habilitar uno excelente después.

La solución de CART (Breiman et al., 1984): crecer un árbol grande T_0 y luego **podar** resolviendo

$$\min_{T \subseteq T_0} \sum_{m=1}^{|T|} \sum_{i: x_i \in R_m} (y_i - \bar{y}_{R_m})^2 + \alpha |T|$$

donde $|T|$ es el número de hojas.

- ¿Les suena la estructura? **Ajuste + castigo por complejidad:** es la misma lógica de la regularización, con α en el papel de λ y “hojas” en el papel de “coeficientes”.
- Al variar α se obtiene una **secuencia anidada** de subárboles; α se elige por **validación cruzada**. El aparato de la sesión 2 aplica sin cambios.

El veredicto sobre el árbol solitario

Lo bueno:

- Interpretable y graficable (se puede *mostrar* a un hacedor de política).
- No linealidades e interacciones automáticas; invariante a transformaciones monótonas de los predictores (¡sin estandarizar!); maneja variables mixtas y faltantes con gracia.

El veredicto sobre el árbol solitario

Lo bueno:

- Interpretable y graficable (se puede *mostrar* a un hacedor de política).
- No linealidades e interacciones automáticas; invariante a transformaciones monótonas de los predictores (¡sin estandarizar!); maneja variables mixtas y faltantes con gracia.

Lo malo (y es grave):

- Predicción **mediocre**: aproximar superficies suaves con escalones cuesta.
- **Varianza altísima**: cambiar pocas observaciones puede alterar el primer corte — y con él, *todo* el árbol de ahí para abajo. Estructura inestable por construcción.

El veredicto sobre el árbol solitario

Lo bueno:

- Interpretable y graficable (se puede *mostrar* a un hacedor de política).
- No linealidades e interacciones automáticas; invariante a transformaciones monótonas de los predictores (¡sin estandarizar!); maneja variables mixtas y faltantes con gracia.

Lo malo (y es grave):

- Predicción **mediocre**: aproximar superficies suaves con escalones cuesta.
- **Varianza altísima**: cambiar pocas observaciones puede alterar el primer corte — y con él, *todo* el árbol de ahí para abajo. Estructura inestable por construcción.

Pregunta para la sala

Tenemos un método flexible, express, de bajo sesgo... y varianza enorme. Con lo visto en el curso, ¿qué harían para bajar la varianza sin sacrificar la flexibilidad?

El veredicto sobre el árbol solitario

Lo bueno:

- Interpretable y graficable (se puede *mostrar* a un hacedor de política).
- No linealidades e interacciones automáticas; invariante a transformaciones monótonas de los predictores (¡sin estandarizar!); maneja variables mixtas y faltantes con gracia.

Lo malo (y es grave):

- Predicción **mediocre**: aproximar superficies suaves con escalones cuesta.
- **Varianza altísima**: cambiar pocas observaciones puede alterar el primer corte — y con él, *todo* el árbol de ahí para abajo. Estructura inestable por construcción.

Pregunta para la sala

Tenemos un método flexible, express, de bajo sesgo... y varianza enorme. Con lo visto en el curso, ¿qué harían para bajar la varianza sin sacrificar la flexibilidad?

Promediar. Si tuviéramos muchas muestras, entrenaríamos muchos árboles y promediaríamos sus predicciones. No tenemos muchas muestras — pero la sesión 2 nos enseñó a fabricarlas: **bootstrap**.

- 1 Árboles de decisión (CART)
- 2 Bagging y Random Forest
- 3 Boosting y XGBoost
- 4 RF vs. boosting: ¿cuál y cuándo?

Bagging: la aritmética de promediar

El punto de partida: si $Z_1, \dots, Z_B$ son i.i.d. con varianza σ^2 ,

$$\text{Var}(\bar{Z}) = \frac{\sigma^2}{B}$$

promediar B predicciones independientes divide la varianza por B .

Bagging: la aritmética de promediar

El punto de partida: si $Z_1, \dots, Z_B$ son i.i.d. con varianza σ^2 ,

$$\text{Var}(\bar{Z}) = \frac{\sigma^2}{B}$$

promediar B predicciones independientes divide la varianza por B .

Bagging (*bootstrap aggregating*; Breiman, 1996):

- 1 Extraer B remuestras bootstrap de los datos.
- 2 Entrenar un árbol **grande, sin podar** en cada una (sesgo bajo, varianza alta — no importa: la varianza la mata el promedio).
- 3 Predecir con el **promedio** (regresión) o el **voto mayoritario** (clasificación):

$$\hat{f}_{bag}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{*b}(x)$$

Bagging: la aritmética de promediar

El punto de partida: si $Z_1, \dots, Z_B$ son i.i.d. con varianza σ^2 ,

$$\text{Var}(\bar{Z}) = \frac{\sigma^2}{B}$$

promediar B predicciones independientes divide la varianza por B .

Bagging (*bootstrap aggregating*; Breiman, 1996):

- 1 Extraer B remuestras bootstrap de los datos.
- 2 Entrenar un árbol **grande, sin podar** en cada una (sesgo bajo, varianza alta — no importa: la varianza la mata el promedio).
- 3 Predecir con el **promedio** (regresión) o el **voto mayoritario** (clasificación):

$$\hat{f}_{bag}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{*b}(x)$$

Pero hay un problema

Los B árboles **no** son independientes: se entrenaron con remuestras de los *mismos* datos. ¿Cuánto promedio nos roba esa correlación? Hagamos la cuenta.

La derivación clave: promediar bajo correlación

Sean $Z_1, \dots, Z_B$ con varianza σ^2 y correlación $\rho > 0$ entre cada par (los árboles se parecen).

La derivación clave: promediar bajo correlación

Sean $Z_1, \dots, Z_B$ con varianza σ^2 y correlación $\rho > 0$ entre cada par (los árboles se parecen).

1. Expandimos la varianza del promedio:

$$\text{Var}(\bar{Z}) = \frac{1}{B^2} \left[\sum_{b=1}^B \text{Var}(Z_b) + \sum_{b \neq b'} \text{Cov}(Z_b, Z_{b'}) \right]$$

La derivación clave: promediar bajo correlación

Sean $Z_1, \dots, Z_B$ con varianza σ^2 y correlación $\rho > 0$ entre cada par (los árboles se parecen).

1. Expandimos la varianza del promedio:

$$\text{Var}(\bar{Z}) = \frac{1}{B^2} \left[\sum_{b=1}^B \text{Var}(Z_b) + \sum_{b \neq b'} \text{Cov}(Z_b, Z_{b'}) \right]$$

2. Sustituimos (B varianzas; $B(B-1)$ covarianzas $= \rho\sigma^2$):

$$\text{Var}(\bar{Z}) = \frac{1}{B^2} \left[B\sigma^2 + B(B-1)\rho\sigma^2 \right]$$

La derivación clave: promediar bajo correlación

Sean $Z_1, \dots, Z_B$ con varianza σ^2 y correlación $\rho > 0$ entre cada par (los árboles se parecen).

1. Expandimos la varianza del promedio:

$$\text{Var}(\bar{Z}) = \frac{1}{B^2} \left[\sum_{b=1}^B \text{Var}(Z_b) + \sum_{b \neq b'} \text{Cov}(Z_b, Z_{b'}) \right]$$

2. Sustituimos (B varianzas; $B(B-1)$ covarianzas $= \rho\sigma^2$):

$$\text{Var}(\bar{Z}) = \frac{1}{B^2} \left[B\sigma^2 + B(B-1)\rho\sigma^2 \right]$$

3. Reorganizamos:

$$\boxed{\text{Var}(\bar{Z}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2}$$

La derivación clave: promediar bajo correlación

Sean $Z_1, \dots, Z_B$ con varianza σ^2 y correlación $\rho > 0$ entre cada par (los árboles se parecen).

1. Expandimos la varianza del promedio:

$$\text{Var}(\bar{Z}) = \frac{1}{B^2} \left[\sum_{b=1}^B \text{Var}(Z_b) + \sum_{b \neq b'} \text{Cov}(Z_b, Z_{b'}) \right]$$

2. Sustituimos (B varianzas; $B(B-1)$ covarianzas $= \rho\sigma^2$):

$$\text{Var}(\bar{Z}) = \frac{1}{B^2} \left[B\sigma^2 + B(B-1)\rho\sigma^2 \right]$$

3. Reorganizamos:

$$\text{Var}(\bar{Z}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$$

- Cuando $B \rightarrow \infty$, el segundo término muere... pero queda $\rho\sigma^2$: **un piso que ningún número de árboles rompe.**
- Dos lecturas: (i) aumentar B **nunca sobreajusta** (solo satura); (ii) el cuello de botella no es B , es ρ . Para ganar más, hay que **descorrelacionar** los

Random Forest = bagging + descorrelación

La idea de Breiman (2001): en cada corte de cada árbol, sortear un subconjunto de **solo m predictores** y elegir el mejor corte *entre esos* ($m \approx \sqrt{p}$ en clasificación; $m \approx p/3$ en regresión).

Random Forest = bagging + descorrelación

La idea de Breiman (2001): en cada corte de cada árbol, sortear un subconjunto de **solo m predictores** y elegir el mejor corte *entre esos* ($m \approx \sqrt{p}$ en clasificación; $m \approx p/3$ en regresión).

¿Por qué funciona?

- Con bagging puro, si existe un predictor **dominante**, casi todos los árboles lo usan en el primer corte $\Rightarrow$ árboles clones $\Rightarrow \rho$ alto $\Rightarrow$ el piso $\rho\sigma^2$ queda alto.

Random Forest = bagging + descorrelación

La idea de Breiman (2001): en cada corte de cada árbol, sortear un subconjunto de **solo m predictores** y elegir el mejor corte *entre esos* ($m \approx \sqrt{p}$ en clasificación; $m \approx p/3$ en regresión).

¿Por qué funciona?

- Con bagging puro, si existe un predictor **dominante**, casi todos los árboles lo usan en el primer corte $\Rightarrow$ árboles clones $\Rightarrow \rho$ alto $\Rightarrow$ el piso $\rho\sigma^2$ queda alto.
- Al sortear candidatos, muchos árboles *ni ven* al dominante en sus primeros cortes: exploran estructuras distintas $\Rightarrow \rho$ **baja** $\Rightarrow$ el piso baja.
- El precio: cada árbol individual empeora un poco (más sesgo). El trade-off vuelve — y empíricamente, bajar ρ gana.

Random Forest = bagging + descorrelación

La idea de Breiman (2001): en cada corte de cada árbol, sortear un subconjunto de **solo m predictores** y elegir el mejor corte *entre esos* ($m \approx \sqrt{p}$ en clasificación; $m \approx p/3$ en regresión).

¿Por qué funciona?

- Con bagging puro, si existe un predictor **dominante**, casi todos los árboles lo usan en el primer corte $\Rightarrow$ árboles clones $\Rightarrow \rho$ alto $\Rightarrow$ el piso $\rho\sigma^2$ queda alto.
- Al sortear candidatos, muchos árboles *ni ven* al dominante en sus primeros cortes: exploran estructuras distintas $\Rightarrow \rho$ **baja** $\Rightarrow$ el piso baja.
- El precio: cada árbol individual empeora un poco (más sesgo). El trade-off vuelve — y empíricamente, bajar ρ gana.

Los hiperparámetros del RF

B : grande y sin miedo (500–1000; no sobreajusta). m : la perilla que importa (CV o defaults). Tamaño mínimo de hoja: controla la profundidad. En R: `ranger`.

Out-of-bag: el error de prueba gratis

¿Recuerdan el 0.632 de la sesión 2? Cada remuestra bootstrap deja fuera $\approx 36.8\%$ de las observaciones.

Out-of-bag: el error de prueba gratis

¿Recuerdan el 0.632 de la sesión 2? Cada remuestra bootstrap deja fuera $\approx 36.8\%$ de las observaciones.

La consecuencia práctica en un bosque:

- Para cada observación i , aproximadamente un tercio de los árboles **no la vieron** al entrenarse.
- **Predicción OOB de i :** promediar (o votar) usando *solo* esos árboles.
- **Error OOB:** el error de esas predicciones sobre toda la muestra — una estimación del error de prueba **sin apartar datos ni correr CV**.

Out-of-bag: el error de prueba gratis

¿Recuerdan el 0.632 de la sesión 2? Cada remuestra bootstrap deja fuera $\approx 36.8\%$ de las observaciones.

La consecuencia práctica en un bosque:

- Para cada observación i , aproximadamente un tercio de los árboles **no la vieron** al entrenarse.
- **Predicción OOB de i :** promediar (o votar) usando *solo* esos árboles.
- **Error OOB:** el error de esas predicciones sobre toda la muestra — una estimación del error de prueba **sin apartar datos ni correr CV**.
- Con B grande, el error OOB es esencialmente equivalente al de LOOCV — gratis, como subproducto del entrenamiento.
- En la práctica: monitorear el error OOB mientras crece B ; cuando se aplanan, ya hay árboles suficientes.

Importancia de variables (y sus trampas)

El bosque pierde la interpretabilidad del árbol único; el sustituto estándar es la **importancia de variables**:

- ❶ **Por impureza:** sumar, sobre todos los cortes de todos los árboles, la reducción de RSS/Gini que aportó cada variable.
- ❷ **Por permutación:** barajar la variable j en las muestras OOB y medir cuánto empeora la predicción.

Importancia de variables (y sus trampas)

El bosque pierde la interpretabilidad del árbol único; el sustituto estándar es la **importancia de variables**:

- 1 **Por impureza:** sumar, sobre todos los cortes de todos los árboles, la reducción de RSS/Gini que aportó cada variable.
- 2 **Por permutación:** barajar la variable j en las muestras OOB y medir cuánto empeora la predicción.

Las trampas (léanlas dos veces):

- La importancia por impureza está **sesgada** hacia variables continuas o con muchas categorías (más umbrales donde cortar = más oportunidades de “lucirse”).

Importancia de variables (y sus trampas)

El bosque pierde la interpretabilidad del árbol único; el sustituto estándar es la **importancia de variables**:

- 1 **Por impureza:** sumar, sobre todos los cortes de todos los árboles, la reducción de RSS/Gini que aportó cada variable.
- 2 **Por permutación:** barajar la variable j en las muestras OOB y medir cuánto empeora la predicción.

Las trampas (léanlas dos veces):

- La importancia por impureza está **sesgada** hacia variables continuas o con muchas categorías (más umbrales donde cortar = más oportunidades de “lucirse”).
- Con predictores **correlacionados**, la importancia se **reparte** entre ellos: una variable crucial puede lucir mediocre porque su gemela absorbió la mitad del crédito.

Importancia de variables (y sus trampas)

El bosque pierde la interpretabilidad del árbol único; el sustituto estándar es la **importancia de variables**:

- 1 **Por impureza:** sumar, sobre todos los cortes de todos los árboles, la reducción de RSS/Gini que aportó cada variable.
- 2 **Por permutación:** barajar la variable j en las muestras OOB y medir cuánto empeora la predicción.

Las trampas (léanlas dos veces):

- La importancia por impureza está **sesgada** hacia variables continuas o con muchas categorías (más umbrales donde cortar = más oportunidades de “lucirse”).
- Con predictores **correlacionados**, la importancia se **reparte** entre ellos: una variable crucial puede lucir mediocre porque su gemela absorbió la mitad del crédito.
- Y la de siempre, elevada al cuadrado: **importancia predictiva $\neq$ efecto causal**. La lección del proxy de la sesión 3 aplica intacta a los bosques.

Importancia de variables (y sus trampas)

El bosque pierde la interpretabilidad del árbol único; el sustituto estándar es la **importancia de variables**:

- 1 **Por impureza:** sumar, sobre todos los cortes de todos los árboles, la reducción de RSS/Gini que aportó cada variable.
- 2 **Por permutación:** barajar la variable j en las muestras OOB y medir cuánto empeora la predicción.

Las trampas (léanlas dos veces):

- La importancia por impureza está **sesgada** hacia variables continuas o con muchas categorías (más umbrales donde cortar = más oportunidades de “lucirse”).
- Con predictores **correlacionados**, la importancia se **reparte** entre ellos: una variable crucial puede lucir mediocre porque su gemela absorbió la mitad del crédito.
- Y la de siempre, elevada al cuadrado: **importancia predictiva $\neq$ efecto causal**. La lección del proxy de la sesión 3 aplica intacta a los bosques.

Uso correcto

Random Forest en la práctica

Por qué es el caballo de batalla:

- Excelente desempeño *out-of-the-box*: casi sin tuning, sin estandarizar, robusto a ruido y a predictores irrelevantes.
- Paraleliza trivialmente (B árboles independientes); error OOB gratis.

Random Forest en la práctica

Por qué es el caballo de batalla:

- Excelente desempeño *out-of-the-box*: casi sin tuning, sin estandarizar, robusto a ruido y a predictores irrelevantes.
- Paraleliza trivialmente (B árboles independientes); error OOB gratis.

Sus límites (para no idolatrar):

- **No extrapola**: predice promedios de hojas $\Rightarrow$ fuera del rango observado de los predictores, devuelve constantes. Fatal para tendencias (¡series de tiempo con crecimiento!).

Random Forest en la práctica

Por qué es el caballo de batalla:

- Excelente desempeño *out-of-the-box*: casi sin tuning, sin estandarizar, robusto a ruido y a predictores irrelevantes.
- Paraleliza trivialmente (B árboles independientes); error OOB gratis.

Sus límites (para no idolatrar):

- **No extrapola**: predice promedios de hojas $\Rightarrow$ fuera del rango observado de los predictores, devuelve constantes. Fatal para tendencias (¡series de tiempo con crecimiento!).
- Superficies muy suaves o efectos estrictamente lineales: un modelo lineal bien regularizado puede ganarle con menos datos.
- Costo en memoria/predicción con B y n grandes.

Random Forest en la práctica

Por qué es el caballo de batalla:

- Excelente desempeño *out-of-the-box*: casi sin tuning, sin estandarizar, robusto a ruido y a predictores irrelevantes.
- Paraleliza trivialmente (B árboles independientes); error OOB gratis.

Sus límites (para no idolatrar):

- **No extrapola**: predice promedios de hojas $\Rightarrow$ fuera del rango observado de los predictores, devuelve constantes. Fatal para tendencias (¡series de tiempo con crecimiento!).
- Superficies muy suaves o efectos estrictamente lineales: un modelo lineal bien regularizado puede ganarle con menos datos.
- Costo en memoria/predicción con B y n grandes.

Teaser de la sesión 6

Un Random Forest cuyas hojas estiman *efectos del tratamiento* en vez de medias de y se llama **causal forest** (Athey & Wager). La maquinaria de hoy es prerequisite directo.

- 1 Árboles de decisión (CART)
- 2 Bagging y Random Forest
- 3 Boosting y XGBoost
- 4 RF vs. boosting: ¿cuál y cuándo?

Boosting: la filosofía opuesta

	Random Forest	Boosting
Árboles	grandes, sin podar	pequeños (<i>weak learners</i>)
Entrenamiento	en paralelo, independientes	en secuencia , corrigiendo al anterior
Agregación	promedio	suma ponderada
Ataca	la varianza	el sesgo (parte rígido)

Boosting: la filosofía opuesta

	Random Forest	Boosting
Árboles	grandes, sin podar	pequeños (<i>weak learners</i>)
Entrenamiento	en paralelo, independientes	en secuencia , corrigiendo al anterior
Agregación	promedio	suma ponderada
Ataca	la varianza	el sesgo (parte rígido)

La forma del modelo final:

$$\hat{f}(x) = \sum_{m=1}^B \nu \hat{T}_m(x)$$

una suma de B árboles pequeños (a menudo de 1–3 cortes), cada uno multiplicado por una **tasa de aprendizaje** ν pequeña.

Boosting: la filosofía opuesta

	Random Forest	Boosting
Árboles	grandes, sin podar	pequeños (<i>weak learners</i>)
Entrenamiento	en paralelo, independientes	en secuencia , corrigiendo al anterior
Agregación	promedio	suma ponderada
Ataca	la varianza	el sesgo (parte rígido)

La forma del modelo final:

$$\hat{f}(x) = \sum_{m=1}^B \nu \hat{T}_m(x)$$

una suma de B árboles pequeños (a menudo de 1–3 cortes), cada uno multiplicado por una **tasa de aprendizaje** ν pequeña.

- Cada árbol aporta un ajuste *modesto* donde el ensamble acumulado todavía se equivoca.
- El lema del boosting: **aprender despacio**. Muchos pasos chicos generalizan mejor que pocos pasos grandes.

El algoritmo, en su versión más simple (regresión)

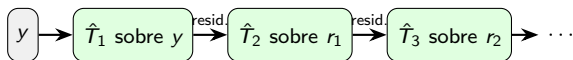
Boosting con pérdida cuadrática:

- ➊ Inicializar $\hat{f}(x) = 0$ y residuos $r_i = y_i$.
- ➋ Para $m = 1, \dots, B$:
 - ▶ Ajustar un árbol pequeño $\hat{T}_m$ a los **residuos actuales** (x_i, r_i) .
 - ▶ Actualizar el modelo: $\hat{f}(x) \leftarrow \hat{f}(x) + \nu \hat{T}_m(x)$.
 - ▶ Actualizar los residuos: $r_i \leftarrow r_i - \nu \hat{T}_m(x_i)$.
- ➌ Devolver $\hat{f}(x) = \sum_m \nu \hat{T}_m(x)$.

El algoritmo, en su versión más simple (regresión)

Boosting con pérdida cuadrática:

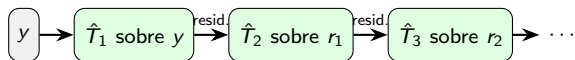
- 1 Inicializar $\hat{f}(x) = 0$ y residuos $r_i = y_i$.
- 2 Para $m = 1, \dots, B$:
 - ▶ Ajustar un árbol pequeño $\hat{T}_m$ a los **residuos actuales** (x_i, r_i) .
 - ▶ Actualizar el modelo: $\hat{f}(x) \leftarrow \hat{f}(x) + \nu \hat{T}_m(x)$.
 - ▶ Actualizar los residuos: $r_i \leftarrow r_i - \nu \hat{T}_m(x_i)$.
- 3 Devolver $\hat{f}(x) = \sum_m \nu \hat{T}_m(x)$.



El algoritmo, en su versión más simple (regresión)

Boosting con pérdida cuadrática:

- 1 Inicializar $\hat{f}(x) = 0$ y residuos $r_i = y_i$.
- 2 Para $m = 1, \dots, B$:
 - ▶ Ajustar un árbol pequeño $\hat{T}_m$ a los **residuos actuales** (x_i, r_i) .
 - ▶ Actualizar el modelo: $\hat{f}(x) \leftarrow \hat{f}(x) + \nu \hat{T}_m(x)$.
 - ▶ Actualizar los residuos: $r_i \leftarrow r_i - \nu \hat{T}_m(x_i)$.
- 3 Devolver $\hat{f}(x) = \sum_m \nu \hat{T}_m(x)$.



- Cada árbol ve **lo que falta por explicar** — no los datos originales.
- ¿Y por qué a esto se le dice *gradient* boosting? Esa es la idea grande que sigue.

La idea grande: descenso de gradiente en el espacio de funciones

Observación de Friedman (2001): el residuo es el gradiente negativo de la pérdida cuadrática respecto a la predicción:

$$-\frac{\partial}{\partial f(x_i)} \frac{1}{2}(y_i - f(x_i))^2 = y_i - f(x_i) = r_i$$

La idea grande: descenso de gradiente en el espacio de funciones

Observación de Friedman (2001): el residuo es el gradiente negativo de la pérdida cuadrática respecto a la predicción:

$$-\frac{\partial}{\partial f(x_i)} \frac{1}{2}(y_i - f(x_i))^2 = y_i - f(x_i) = r_i$$

Ajustar árboles a los residuos = dar pasos de **descenso de gradiente**, donde el “parámetro” es la función f entera y ν es el tamaño del paso.

La idea grande: descenso de gradiente en el espacio de funciones

Observación de Friedman (2001): el residuo es el gradiente negativo de la pérdida cuadrática respecto a la predicción:

$$-\frac{\partial}{\partial f(x_i)} \frac{1}{2}(y_i - f(x_i))^2 = y_i - f(x_i) = r_i$$

Ajustar árboles a los residuos = dar pasos de **descenso de gradiente**, donde el “parámetro” es la función f entera y ν es el tamaño del paso.

La generalización inmediata: para *cualquier* pérdida L , ajustar cada árbol al **pseudo-residuo**

$$r_{im} = - \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)} \right]_{f=\hat{f}_{m-1}}$$

La idea grande: descenso de gradiente en el espacio de funciones

Observación de Friedman (2001): el residuo es el gradiente negativo de la pérdida cuadrática respecto a la predicción:

$$-\frac{\partial}{\partial f(x_i)} \frac{1}{2}(y_i - f(x_i))^2 = y_i - f(x_i) = r_i$$

Ajustar árboles a los residuos = dar pasos de **descenso de gradiente**, donde el “parámetro” es la función f entera y ν es el tamaño del paso.

La generalización inmediata: para *cualquier* pérdida L , ajustar cada árbol al **pseudo-residuo**

$$r_{im} = - \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)} \right]_{f=\hat{f}_{m-1}}$$

- Pérdida absoluta $\Rightarrow r_{im} = \text{sign}(y_i - \hat{f}(x_i))$: robusta a atípicos.
- Pérdida logística $\Rightarrow r_{im} = y_i - \hat{p}(x_i)$: boosting para **clasificación** (¡lo reencontraremos en la sesión 5!).

La idea grande: descenso de gradiente en el espacio de funciones

Observación de Friedman (2001): el residuo es el gradiente negativo de la pérdida cuadrática respecto a la predicción:

$$-\frac{\partial}{\partial f(x_i)} \frac{1}{2}(y_i - f(x_i))^2 = y_i - f(x_i) = r_i$$

Ajustar árboles a los residuos = dar pasos de **descenso de gradiente**, donde el “parámetro” es la función f entera y ν es el tamaño del paso.

La generalización inmediata: para *cualquier* pérdida L , ajustar cada árbol al **pseudo-residuo**

$$r_{im} = - \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)} \right]_{f=\hat{f}_{m-1}}$$

- Pérdida absoluta $\Rightarrow r_{im} = \text{sign}(y_i - \hat{f}(x_i))$: robusta a atípicos.
- Pérdida logística $\Rightarrow r_{im} = y_i - \hat{p}(x_i)$: boosting para **clasificación** (¡lo reencontraremos en la sesión 5!).

Por qué esto es elegante

Las tres perillas del boosting (y cómo interactúan)

- 1 B — **número de árboles**. A diferencia del RF, aquí B grande sí **sobreajusta**: el ensamble sigue ajustando residuos hasta memorizar el ruido. B se elige por CV o *early stopping* (parar cuando el error de validación deja de bajar).

Las tres perillas del boosting (y cómo interactúan)

- 1 B — **número de árboles**. A diferencia del RF, aquí B grande sí **sobreajusta**: el ensamble sigue ajustando residuos hasta memorizar el ruido. B se elige por CV o *early stopping* (parar cuando el error de validación deja de bajar).
- 2 ν — **tasa de aprendizaje** (típico 0.01–0.1). Más pequeña = pasos más cortos = mejor generalización... a cambio de necesitar más árboles: ν y B se compensan (bajar ν $10\times \approx$ subir B $10\times$).

Las tres perillas del boosting (y cómo interactúan)

- 1 B — **número de árboles**. A diferencia del RF, aquí B grande sí **sobreajusta**: el ensamble sigue ajustando residuos hasta memorizar el ruido. B se elige por CV o *early stopping* (parar cuando el error de validación deja de bajar).
- 2 ν — **tasa de aprendizaje** (típico 0.01–0.1). Más pequeña = pasos más cortos = mejor generalización. . . a cambio de necesitar más árboles: ν y B se compensan (bajar ν $10\times \approx$ subir B $10\times$).
- 3 d — **profundidad de cada árbol**. Controla el **orden de interacción**: $d = 1$ (*stumps*) = modelo aditivo puro; $d = 2$ permite interacciones de a pares; rara vez conviene $d > 6$.

Las tres perillas del boosting (y cómo interactúan)

- 1 B — **número de árboles**. A diferencia del RF, aquí B grande sí **sobreajusta**: el ensamble sigue ajustando residuos hasta memorizar el ruido. B se elige por CV o *early stopping* (parar cuando el error de validación deja de bajar).
- 2 ν — **tasa de aprendizaje** (típico 0.01–0.1). Más pequeña = pasos más cortos = mejor generalización. . . a cambio de necesitar más árboles: ν y B se compensan (bajar ν $10\times \approx$ subir B $10\times$).
- 3 d — **profundidad de cada árbol**. Controla el **orden de interacción**: $d = 1$ (*stumps*) = modelo aditivo puro; $d = 2$ permite interacciones de a pares; rara vez conviene $d > 6$.

La disciplina del boosting

Es el método más poderoso de esta sesión **y** el más fácil de arruinar: exige validación cuidadosa (sesión 2) y paciencia con el tuning. El RF perdona; el boosting no.

XGBoost: la implementación que se comió al mundo

Chen & Guestrin (2016) — *gradient boosting* + ingeniería + una idea estadística:

XGBoost: la implementación que se comió al mundo

Chen & Guestrin (2016) — *gradient boosting* + ingeniería + una idea estadística:

- **Objetivo con regularización explícita:** cada árbol se elige minimizando

$$\sum_i L(y_i, \hat{f}(x_i)) + \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

donde T es el número de hojas y w_j sus valores: **Ridge y poda dentro del propio boosting** — las sesiones 2 y hoy, fundidas en una fórmula.

XGBoost: la implementación que se comió al mundo

Chen & Guestrin (2016) — *gradient boosting* + ingeniería + una idea estadística:

- **Objetivo con regularización explícita:** cada árbol se elige minimizando

$$\sum_i L(y_i, \hat{f}(x_i)) + \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

donde T es el número de hojas y w_j sus valores: **Ridge y poda dentro del propio boosting** — las sesiones 2 y hoy, fundidas en una fórmula.

- **Expansión de segundo orden** de la pérdida (gradiente y hessiano): pasos tipo Newton, más precisos que el gradiente puro.

XGBoost: la implementación que se comió al mundo

Chen & Guestrin (2016) — *gradient boosting* + ingeniería + una idea estadística:

- **Objetivo con regularización explícita:** cada árbol se elige minimizando

$$\sum_i L(y_i, \hat{f}(x_i)) + \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

donde T es el número de hojas y w_j sus valores: **Ridge y poda dentro del propio boosting** — las sesiones 2 y hoy, fundidas en una fórmula.

- **Expansión de segundo orden** de la pérdida (gradiente y hessiano): pasos tipo Newton, más precisos que el gradiente puro.
- *Column subsampling* (la idea del RF, prestada), manejo nativo de faltantes, paralelización por histogramas.

XGBoost: la implementación que se comió al mundo

Chen & Guestrin (2016) — *gradient boosting* + ingeniería + una idea estadística:

- **Objetivo con regularización explícita:** cada árbol se elige minimizando

$$\sum_i L(y_i, \hat{f}(x_i)) + \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

donde T es el número de hojas y w_j sus valores: **Ridge y poda dentro del propio boosting** — las sesiones 2 y hoy, fundidas en una fórmula.

- **Expansión de segundo orden** de la pérdida (gradiente y hessiano): pasos tipo Newton, más precisos que el gradiente puro.
- *Column subsampling* (la idea del RF, prestada), manejo nativo de faltantes, paralelización por histogramas.
- **Primos:** LightGBM (histogramas agresivos, crecimiento por hoja: más veloz en n grande) y CatBoost (categóricas de alta cardinalidad sin *dummies*).
- El resultado: la familia que domina las competencias de Kaggle en datos **tabulares** desde hace una década — redes incluidas.

- 1 Árboles de decisión (CART)
- 2 Bagging y Random Forest
- 3 Boosting y XGBoost
- 4 RF vs. boosting: ¿cuál y cuándo?

Cara a cara

	Random Forest	Boosting (XGBoost)
Estrategia	bajar varianza promediando	bajar sesgo sumando en secuencia
B grande	inofensivo (satura)	sobreajusta (early stopping)
Tuning	mínimo (m)	delicado (ν , d , B , regularización)
Error sin CV	OOB gratis	no (validación aparte)
Paralelización	trivial	parcial (dentro de cada árbol)
Desempeño típico	excelente, robusto	el mejor , si se afina bien

Cara a cara

	Random Forest	Boosting (XGBoost)
Estrategia	bajar varianza promediando	bajar sesgo sumando en secuencia
B grande	inofensivo (satura)	sobreajusta (early stopping)
Tuning	mínimo (m)	delicado (ν , d , B , regularización)
Error sin CV	OOB gratis	no (validación aparte)
Paralelización	trivial	parcial (dentro de cada árbol)
Desempeño típico	excelente, robusto	el mejor , si se afina bien

Regla práctica para sus proyectos:

- 1 Empezar con una **regresión regularizada** (línea base interpretable; sesiones 2–3).
- 2 **Random Forest**: el salto no lineal con mínimo esfuerzo; su OOB dice si valió la pena.
- 3 **XGBoost** afinado con CV: cuando el objetivo es exprimir la última gota de desempeño.
- 4 Y reportar los tres en la misma tabla, evaluados en el **mismo** conjunto de prueba.

¿Y en economía?

Dónde los verán trabajando:

- **Predicción de pobreza e ingresos** para focalización: RF/boosting sobre encuestas y registros — el estándar en los *proxy means tests* modernos.

¿Y en economía?

Dónde los verán trabajando:

- **Predicción de pobreza e ingresos** para focalización: RF/boosting sobre encuestas y registros — el estándar en los *proxy means tests* modernos.
- **Riesgo de crédito**: XGBoost es, con distancia, el método más desplegado en *scoring* bancario y *fintech* (la aplicación de esta semana).

¿Y en economía?

Dónde los verán trabajando:

- **Predicción de pobreza e ingresos** para focalización: RF/boosting sobre encuestas y registros — el estándar en los *proxy means tests* modernos.
- **Riesgo de crédito**: XGBoost es, con distancia, el método más desplegado en *scoring* bancario y *fintech* (la aplicación de esta semana).
- **Precios hedónicos e inmobiliarios**: interacciones ubicación $\times$ características que un modelo lineal no captura sin ayuda.

¿Y en economía?

Dónde los verán trabajando:

- **Predicción de pobreza e ingresos** para focalización: RF/boosting sobre encuestas y registros — el estándar en los *proxy means tests* modernos.
- **Riesgo de crédito**: XGBoost es, con distancia, el método más desplegado en *scoring* bancario y *fintech* (la aplicación de esta semana).
- **Precios hedónicos e inmobiliarios**: interacciones ubicación $\times$ características que un modelo lineal no captura sin ayuda.
- **Insumo de otros métodos**: en el *double ML* de la sesión 6, los bosques y el boosting serán los motores de predicción dentro de un procedimiento causal ortogonalizado — la predicción al servicio de β .

¿Y en economía?

Dónde los verán trabajando:

- **Predicción de pobreza e ingresos** para focalización: RF/boosting sobre encuestas y registros — el estándar en los *proxy means tests* modernos.
- **Riesgo de crédito**: XGBoost es, con distancia, el método más desplegado en *scoring* bancario y *fintech* (la aplicación de esta semana).
- **Precios hedónicos e inmobiliarios**: interacciones ubicación $\times$ características que un modelo lineal no captura sin ayuda.
- **Insumo de otros métodos**: en el *double ML* de la sesión 6, los bosques y el boosting serán los motores de predicción dentro de un procedimiento causal ortogonalizado — la predicción al servicio de β .

La advertencia de siempre, por tercera vez

RF y boosting producen **predicciones e importancias**, no efectos causales. Interpretarlos causalmente sin diseño es el mismo pecado de la sesión 3, con mejor maquillaje.

Recap

- 1 **CART**: particiones binarias greedy; Gini/entropía para crecer, poda costo-complejidad (α por CV) para generalizar. Interpretable, flexible... y de varianza altísima.
- 2 **Bagging**: promediar árboles sobre remuestras bootstrap. La cuenta clave: $\text{Var}(\bar{Z}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$ — aumentar B nunca sobreajusta, pero el piso lo pone ρ .
- 3 **Random Forest**: descorrelacionar sorteando $m \approx \sqrt{p}$ candidatos por corte; error **OOB** gratis; importancias útiles pero sesgadas, repartidas por correlación y **jamás causales**.
- 4 **Boosting**: árboles pequeños en secuencia sobre pseudo-residuos = **descenso de gradiente en el espacio de funciones**; funciona con cualquier pérdida; aprender despacio (ν chico, B por early stopping).
- 5 **XGBoost**: boosting con regularización explícita ($\gamma T + \frac{\lambda}{2}\|w\|^2$) y segundo orden — el dominador de los datos tabulares.
- 6 **Flujo de trabajo**: base lineal $\rightarrow$ RF $\rightarrow$ XGBoost, todos evaluados en la misma prueba.

Preparación para la próxima clase...

Para aprovechar al máximo la próxima sesión

Lecturas obligatorias de esta semana:

- ISLR, cap. 8 completo.

Guía: es el capítulo que mejor mapea a la sesión de hoy; las figuras 8.1–8.6 son las de clase.

- Breiman (2001), *Random Forests*.

Guía: secciones 1–2; el resto, para quien quiera las cotas formales.

Recomendadas: Friedman (2001; secs. 1–4); Chen & Guestrin (2016); Biau & Scornet (2016, survey amable de RF).

Para aprovechar al máximo la próxima sesión

Lecturas obligatorias de esta semana:

- ISLR, cap. 8 completo.

Guía: es el capítulo que mejor mapea a la sesión de hoy; las figuras 8.1–8.6 son las de clase.

- Breiman (2001), *Random Forests*.

Guía: secciones 1–2; el resto, para quien quiera las cotas formales.

Recomendadas: Friedman (2001; secs. 1–4); Chen & Guestrin (2016); Biau & Scornet (2016, survey amable de RF).

Próxima sesión: clasificación y métricas de desempeño — logística por máxima verosimilitud, k -NN, SVM, y la parte que decide todo en la práctica: **ROC/AUC, umbrales por costos y desbalanceo de clases**. Repasen máxima verosimilitud de Econometría II.

Para aprovechar al máximo la próxima sesión

Lecturas obligatorias de esta semana:

- ISLR, cap. 8 completo.

Guía: es el capítulo que mejor mapea a la sesión de hoy; las figuras 8.1–8.6 son las de clase.

- Breiman (2001), *Random Forests*.

Guía: secciones 1–2; el resto, para quien quiera las cotas formales.

Recomendadas: Friedman (2001; secs. 1–4); Chen & Guestrin (2016); Biau & Scornet (2016, survey amable de RF).

Próxima sesión: clasificación y métricas de desempeño — logística por máxima verosimilitud, k -NN, SVM, y la parte que decide todo en la práctica: **ROC/AUC, umbrales por costos y desbalanceo de clases**. Repasen máxima verosimilitud de Econometría II.

Aplicación de esta semana en R: riesgo de crédito — árbol vs. Random Forest (`rpart`, `ranger`) vs. XGBoost (`xgboost`), comparados por CV.

Referencias esenciales

- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning* (2. ed.). Springer. Cap. 8. [ISLR]
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2. ed.). Springer. Caps. 9, 10 y 15. [ESL]
- Breiman, L., Friedman, J., Olshen, R., & Stone, C. (1984). *Classification and Regression Trees*. Wadsworth.
- Breiman, L. (1996). Bagging Predictors. *Machine Learning*, 24(2), 123–140.
- Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
- Biau, G., & Scornet, E. (2016). A Random Forest Guided Tour. *TEST*, 25(2), 197–227.
- Freund, Y., & Schapire, R. (1997). A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting. *JCSS*, 55(1), 119–139.
- Friedman, J. (2001). Greedy Function Approximation: A Gradient Boosting Machine. *Annals of Statistics*, 29(5), 1189–1232.
- Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *KDD '16*, 785–794.
- Wager, S., & Athey, S. (2018). Estimation and Inference of Heterogeneous Treatment Effects using Random Forests. *JASA*, 113(523), 1228–1242.